

Teil II

Programmierung

3. Voraussetzungen

- Programmiersprachen
- Java-Umgebung
- Java API

4. Quelltext

5. Pakete, Klassen, Variablen

6. Methoden

Informatik

Informatik: Wissenschaft der **systematischen** Verarbeitung von Informationen, insb. der **automatischen** mittels Computern

Teilgebiete der **Informatik:**

- **Theoretische Informatik:** logisch-mathematische Prinzipien
- **Praktische Informatik:** Fokus auf Software
(**Software Engineering**, Datenbanken, Betriebssysteme, ...)
- **Technische Informatik:** Fokus auf Hardware
(Computer, -bausteine, Netzwerke, Ein-/Ausgabegeräte, ...)

Aufgabenstellungen des **Software Engineering** u.a.:

- **Entwicklung** von Software (inkl. Planung, Management)
- **Untersuchung** von Software
- **Auswahl** von Software

Softwareentwicklung

wesentliche Phasen im **Softwareentwicklungsprozess**:

- ➊ **Analyse**: „was genau ist das Problem?“ → **Anforderungen**
- ➋ **Entwurf**: **Modell** des Problems, Wahl einer **Lösung**
- ➌ **Programmieren**: **Formalisierung** von Modell und Lösung in **Programmiersprache(n)** → **Programm(teile)**
- ➍ **Testen**: „erfüllt das Programm die Anforderungen?“
- ➎ **Wartung**: Korrekturen, Anpassung an neue Anforderungen

für kleinere Aufgaben: „**Programmieren**“ $\hat{=}$ gesamter Prozess

Programmiersprachen

Sprachen

wie **Menschen** Probleme und Lösungen beschreiben:

„... Die Schokolade wird im Wasserbad geschmolzen. Die Eier werden getrennt. Das Eiweiß wird steif geschlagen und kühl gestellt, ebenso die Sahne. Das Eigelb wird mit dem Zucker schaumig geschlagen, dann die Schokolade langsam ...“

- **verständlich** für Menschen
- ungenau (wie kühl? mit Zucker Eigelb schlagen – wirklich?)

wie **Computer** Probleme und Lösungen beschreiben:

„...1001011110110101100001100011110101100...“

- unverständlich für Menschen
- **exakt** (Bedeutung von 0/1 jeweils genau festgelegt)

Programmiersprachen

Menschen sind **intelligent**, aber **unkonzentriert**, **ungeduldig**, **langsam**:

→ unproduktive, fehlerhafte, begrenzte Arbeit in Computersprache

Computer sind **konzentriert**, **geduldig**, **schnell**, aber **unintelligent**:

→ kein Verständnis natürlicher Sprache

(bzw. kaum und nur unter hohem Aufwand)

Programmiersprache: Kompromiss **natürliche**/Maschinensprache

- **verständlich** für Menschen: (noch) **effektiv** schreibbar/lesbar
- **exakt**: für Computer (schon) **eindeutig** übersetzbar/ausführbar

Elemente von Programmiersprachen

Programmiersprachen sind **reduziert** auf **essentielle Sprachmittel**, die (nur) **Strukturen** und **Verhalten** beschreiben.

bereits bekannte **Sprachmittel in Java**:

- **Struktur**: Klasse, Typ, Wert, Variable
- **Verhalten**: Methode, Anweisung, Kontrollstruktur, Ausdruck

abstraktere Unterscheidung auf **Modellebene** zwischen:

- **Datenstrukturen**: Darstellungen von Daten (→ Klassenimpl.)
- **Algorithmen**: Vorgehensweisen auf jenen (→ Methodenimpl.)

→ lassen sich **unabhängig von der Programmiersprache** beschreiben

Programmierparadigmen

Programmierparadigmen (**Sprachfamilien**) sind charakterisiert durch **wesentliche Sprachmittel** – Java ist

- **imperativ**: hat Zustände (Variablen, Zuweisungen, Schleifen)
- **objektorientiert**: Code in Klassen aufgeteilt (Näheres später)

andere Programmierparadigmen:

- **funktional** (vs. **imperativ**): zustandsfrei, Programm als Ausdruck
- **prozedural** (vs. **obj.or.**): in Prozeduren (~ Methoden) aufgeteilt
- ...

meist liegen **Mischformen** vor (**Multiparadigmen-Sprachen**)

Auswahl an Programmiersprachen

Java (1995 von Sun, heute Oracle, vorgestellt):

- **imperativ** und **objektorientiert**
- **universell** (**general purpose**):
allgemein einsetzbar (kein bestimmtes Anwendungsgebiet)
- **portabel**: auf verschiedenen Systemen übersetzbar/ausführbar
- **verbreitet**: **viele Lösungsbausteine vorhanden**
- „**einfach**“ (im Vergleich zu anderen Sprachen)

weitere universelle Sprachen (aus Hunderten):

- „schöner“, einfacher(?): **Python, Ruby, Kotlin, Smalltalk, ...**
- technischer, komplexer: **C++, Rust, Perl, ...**
- **funktional**: **Scala, Haskell, ML, LISP, ...**
- **prozedural**: **C, FORTRAN, Pascal, ...**
- ...

Inhalte dieses Kurses (u.a.)

- Grundkonzepte der **imperativen Programmierung**
- Grundkonzepte der **objektorientierten Programmierung**
- **sinnvolle Nutzung** der Konzepte
- ... am Beispiel der **Programmiersprache Java**
→ aber **keine umfassende Behandlung** von Java!
- **Prinzipien der Programmausführung** auf Rechnern
- **formale Beschreibung** von Problemen
- **Entwurf und Umsetzung** von Lösungen
- Prüfen der **Korrektheit** von Lösungen

Java-Umgebung

Erstellung

Wiederholung:

Java-Klasse X wird in **Textdatei** $X.java$ erstellt

Textdatei: rein zeichenbasiert, keine Formatierung

→ kann in elementarem **Texteditor** angezeigt / bearbeitet werden

Erstellung kann durch **Entwicklungswerkzeuge** unterstützt werden
(wird aber für einfache Programme dadurch eher komplizierter)

Hinweis: Dateien/Zeichen sollten im Format **UTF-8** codiert werden
(unterstützt Sonderzeichen in portabler Weise)

Übersetzung

Java-**Quellcode** in Datei `X.java` wird von **Übersetzer (Compiler)** in **Bytecode** in Datei `X.class` überführt:

```
$ ls                                MS CMD: dir
... X.java ...
$ javac X.java
$ ls
... X.class X.java ...
```

im Allgemeinen: **Übersetzung** des **Quellcodes** in `X.java`

- von „höherer“ (für Menschen lesbarer)
- in „niedere“ (maschinennähere) Sprache

Hinweis: folgende Option zeigt Arbeitsschritte des Compilers

```
$ javac -verbose X.java
```

Bytecode

Bytecode nicht lesbar, kann durch **Disassembler** angezeigt werden:

```
$ javap -c Hallo.class
Compiled from "Hallo.java"
public class Hallo {
    ...
    public static void main(java.lang.String[]);
        Code:
            0: getstatic      #2 // Field java/lang/System.out...
            3: ldc            #3 // String Hallo Welt!
            5: invokevirtual #4 // Method java/io/PrintStream...
            8: return
}
```

Bytecode wird **normalerweise nicht betrachtet!**

Programm in main

ausführbares (Java-) **Programm** X besteht aus **Bytecode**

- einer Klasse X mit einer Klassenmethode

```
public static void main(String[] args)
```

- und aller Klassen, die von X verwendet werden
- und aller Klassen, die von diesen verwendet werden

(viele verwendete Klassen stammen aus der **Java-Umgebung**)

weitere main-Methoden in anderen Klassen möglich – Idee dahinter:

- (große!) Sammlung von **Lösungsklassen** (ohne main)
- **Anwendungs-/Testklassen** (i.W. nur main) als Startpunkte
z.B. nur Aufruf von: Eingabe – Berechnung – Ausgabe
- **verschiedene Anwendungen/Tests** nutzen **selbe Lösungsklassen**

Ausführung

```
$ java X
```

(Java-) **Interpreter** führt „Programm *X*“ aus, also **Bytecode**

- der **main-Methode** in **Klasse *X*** (in Datei *X.class*) und
- der von dort ggf. **aufgerufenen Methoden** in **anderen Klassen**

Interpreter: Quellcode wird **bei Laufzeit schrittweise(!)**

- 1 gelesen
- 2 in Maschinencode umgewandelt
- 3 ausgeführt

java nur **Front-End**-Kommando: Interpretation eigentlich in **JVM**

JVM (Java Virtual Machine)

JVM („Java Virtual Machine“):

- ist virtueller „Rechner“ mit Bytecode als dessen Maschinencode
- interpretiert diesen auf dem physischen Rechner
- dazu plattformspezifische JVM-Installation erforderlich

Motivation für den Einsatz der JVM:

- Übersetzung Quellcode → Bytecode: 1 Entwickler:in
(aufwendig, fatale Fehler möglich)
- Bytecode maschinennah, aber noch portabel
⇒ wenn einmal erzeugt, (fast) überall ausführbar
- Interpretation Bytecode → Masch.code: viele Anwender:innen
(einfach, kein „Absturz“ möglich!)

Compiler vs. Interpreter

Compiler	Interpreter
produziert Datei in Zwischensprache, ggf. Maschinsprache	produziert Maschinsprache zur sofortigen Ausführung
Resultat ggf. plattformabhängig	überall ausführbar
wird einmal angewendet	pro Ausführung anzuwenden
betrachtet gesamte Datei	betrachtet Anweisungen schrittweise, in Reihenfolge
restrukturiert/optimiert	folgt gegebenem Ablauf
aufwendig, Resultat effizient	sofort verfügbar, ggf. ineffizient
produziert statischen Ablauf	Ablauf an Situation, z.B. Nutzerwünsche (Dialog) anpassbar
gut bei wiederholter Ausführung desselben Codes	gut bei einmaliger (insb. interaktiver) Ausführung

Alternative Verarbeitungsmodelle

Grundmodelle der Verarbeitung von Programmiersprachen:

- **reine Kompilierung:** gesamter Code wird aus Prog.sprache direkt in Maschinensprache übersetzt, später ausgeführt
(C, C++, Go, Rust, Haskell, ...)
- **reine Interpretation:** Code wird direkt aus Programmiersprache stückweise gelesen und jeweils sofort ausgeführt
(Python, Ruby, PHP, Perl, JavaScript, ...)
- **Kombination:** gesamter Code wird aus Programmiersprache in Zwischensprache (d.h. für eine VM) übersetzt, später aus dieser stückweise gelesen und jeweils sofort ausgeführt
(Java, Scala, Kotlin, Groovy, Clojure, ...)

es gibt **viele Varianten**, in manchen Sprachen **mehrere Modelle**

JRE und JDK

JRE („**Java Runtime Environment**“ – Laufzeitumgebung):

- benötigt, um vorhandenen Java-Bytecode **auszuführen**
- benötigt von Java-**Anwendern**
- enthält **JVM** und (Bytecode der) **Java API** (nächstes Thema)

JDK („**Java Development Kit**“ – Entwicklungsbausatz)

- benötigt, um Java-Code zu **entwickeln**
- benötigt von Java-**Entwicklern**
- enthält u.a. **JRE**, **Übersetzer** und **Interpreter**

zur Struktur siehe **Java (8) Platform Documentation**:

<https://docs.oracle.com/javase/8/docs/>

Java API

API

API (Application Program Interface) einer Klassensammlung: für Anwendungen wesentliche Eigenschaften der Klassen und Methoden

Verwendung einer Methode erfordert Kenntnis von deren:

- **Kopf** mit Rückgabe- und Parametertypen (gemäß Deklaration) (→ **Syntax** und **Typisierung**)
- **Wirkung** (gemäß Dokumentation) (→ **Semantik**)

diese Angaben sind in der **API-Spezifikation** festgelegt (siehe „Dokumentation“ in Moodle)

→ nicht verwechseln mit **Sprachspezifikation** (legt die Sprache fest)

Java-API ist **sehr umfangreich**, im Folgenden **elementare Beispiele**

Math-API

Klasse **Math** (eigentlich: `java.lang.Math`)

bekannt: Klassenmethoden `abs` / `min` / `max`

außerdem:

- Klassenmethode `double Math.sqrt(double x)` (Wurzel),
entsprechend `sin` / `cos` / `tan` / ... („Taschenrechner“)
- Gleitkommazahlvariable `double Math.PI` = 3.14159265...
- Klassenmethode `double Math.random()`:
liefert pro Aufruf einen neuen **Zufallswert** im Bereich `[0.0; 1.0[`
(genauer sind es **Pseudozufallswerte**:
bei **jeder Programmausführung** entsteht **selbe Wertfolge**)
- Methoden zur Abschätzung/Behandlung von **Rundungsfehlern**

String-API

wichtige Klasse: **String** (eigentlich: `java.lang.String`)

Instanzmethoden (für String-Werte `s1`, `s2` und int-Werte `i`, `j`):

Aufruf	Typ	Verhalten
<code>s1.length()</code>	<code>int</code>	Länge von <code>s1</code> (Anz. Zeichen)
<code>s1.equals(s2)</code>	<code>boolean</code>	sind <code>s1</code> und <code>s2</code> gleich ?
<code>s1.concat(s2)</code>	<code>String</code>	wie <code>s1 + s2</code>
<code>s1.contains(s2)</code>	<code>boolean</code>	ist <code>s2</code> in <code>s1</code> enthalten ?
<code>s1.substring(i,j)</code>	<code>String</code>	Teilzeichenkette ab Position <code>i</code> bis vor(!) Position <code>j</code>
<code>s1.substring(i)</code>	<code>String</code>	<code>s1.substring(i,s1.length())</code>

Zählung der Position beginnt mit 0 (erstes Zeichen), nicht mit 1!

letzte Position also: `s.length() - 1`

```
"Hallo Welt!".substring(4, 8) // ergibt: "o We"
```

Übung: Zeichenkette.verkuerze

Aufgabenblatt 04, Aufgabe 05

Zeichenketten und Zeichen

String bietet **Instanzmethoden** für **Zugriff auf Zeichen** (char);
für String-Wert s, char-Wert c, int-Wert i z.B.:

Aufruf	Typ	Verhalten
<code>s.charAt(i)</code>	char	Zeichen an Position i in s
<code>s.indexOf(c)</code>	int	erste Position von c in s
<code>s.indexOf(c,i)</code>	int	erste Pos. von c in s, ab Pos. i
<code>s.lastIndexOf(c)</code>	int	letzte Position von c in s

`s.indexOf` und `s.lastIndexOf` liefern -1, falls c nicht in s

Zeichenketten und Zeichen: Beispiel

```
1 String s = "Hallo Welt!";
2
3 char c1 = s.charAt(0);           // 'H'
4 char c2 = s.charAt(5);           // ' '
5 char c2 = s.charAt(s.length() - 1); // '!'
6
7 int g = s.indexOf('4');           // -1
8 int i = s.indexOf('l');           // 2
9 int h = s.indexOf('l', 4);        // 8
10 int j = s.lastIndexOf('l');       // 8
11 String sub = s.substring(i, j);   // "llo We"
```

oder statt der Zeilen 8, 10, 11 kürzer:

```
String sub = s.substring(s.indexOf('l'), s.lastIndexOf('l'));
```

Zeichenketten und Zeichen: istGleich

length und charAt ausreichend zur Implementierung der anderen String-Methoden, z.B. equals:

```
2  public static boolean istGleich(String s1, String s2) {
3      if (s1.length() != s2.length()) {
4          return false;
5      }
6      int i = 0;
7      while (i < s1.length()) {
8          if (s1.charAt(i) != s2.charAt(i)) {
9              return false;
10         }
11         ++i;
12     }
13     return true;
14 }
```

Übung: Zeichenkette 2

Aufgabenblatt 04, Aufgabe 06

Lesen von Zeichen

keine Scanner-Methode zum **Lesen einzelner Zeichen**

stattdessen:

- 1 Lesen einer Zeichenkette (hat Länge 1, wenn nur 1 Zeichen)
- 2 Zugriff auf das erste Zeichen der Kette

Umsetzung (ignoriert weitere Zeichen der Zeichenkette):

```
String s = sc.next();  
char c = s.charAt(0);
```

oder kürzer:

```
char c = sc.next().charAt(0);
```

Methoden auf Zeichen

Klasse **Character** (eigentlich `java.lang.Character`) bietet **Klassenmethoden** zur Verarbeitung von Zeichen (Typ `char`):

Aufruf: <code>Character. ...</code>	Typ	Ergebnis
<code>isLetter(c)</code>	<code>boolean</code>	ist c ein Buchstabe?
<code>isUpperCase(c)</code>	<code>boolean</code>	ist c ein Großbuchstabe?
<code>isLowerCase(c)</code>	<code>boolean</code>	ist c ein Kleinbuchstabe?
<code>isDigit(c)</code>	<code>boolean</code>	ist c eine Ziffer?
<code>isLetterOrDigit(c)</code>	<code>boolean</code>	ist c Buchstabe/Ziffer?
<code>isWhitespace(c)</code>	<code>boolean</code>	ist c ein White Space?
<code>toUpperCase(c)</code>	<code>char</code>	c als Großbuchstabe
<code>toLowerCase(c)</code>	<code>char</code>	c als Kleinbuchstabe

`Character.to...()` liefern c, wenn c kein Klein-/Großbuchstabe

Beispiel: Umwandlung Klein- ↔ Großbuchstaben

```
3 public static String umwandlung(String s) {
4     String r = "";
5     int i = 0;
6     while (i < s.length()) {
7         char c = s.charAt(i);
8         if (Character.isLowerCase(c)) {
9             r += Character.toUpperCase(c);
10        } else {
11            //if (Character.isUpperCase(c)) { // ueberfluessig
12            r += Character.toLowerCase(c);
13            //} else { r += c; }
14        }
15        ++i;
16    }
17    return r;
18 }
```

Zeichen.java

Beispiel: Bestätigung „[Jj]a/[Nn]ein“

```
1 import java.util.Scanner;
  // ...
19 public static boolean abfrage(Scanner sc, String frage) {
20     String prompt = frage + " ([Jj]a/[Nn]ein)";
21     System.out.println(prompt);
22     while (sc.hasNext()) {
23         char c = Character.toLowerCase(sc.next().charAt(0));
24         sc.nextLine(); // Rest der Eingabezeile ignorieren
25         if (c == 'j') {
26             return true;
27         }
28         if (c == 'n') {
29             return false;
30         }
31         System.out.println(prompt);
32     }
33     return false;
34 }
```



Beispiele: Anwendung

```
1 import java.util.Scanner;
2 public class ZeichenTest {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         if (Zeichen.abfrage(sc, "Bereit zu starten?")) {
6             while (sc.hasNext()) {
7                 String s = sc.nextLine();
8                 System.out.println(Zeichen.umwandlung(s));
9             }
10        }
11        sc.close();
12    }
13 }
```

ZeichenTest.java

Übung: Zeichenkette 2

Aufgabenblatt 04, Aufgabe 07

Traits-/Wrapper-Klassen

entsprechend zu `Character`:

Klassen `Boolean`, `Integer`, `Double`, ...

(eigentlich `java.lang.Boolean`, `java.lang.Integer`, ...)

bieten für den entsprechenden primitiven Typ `boolean`, `int`, `double`, ...:

- **Klassenmethoden** auf Werten des primitiven Typs
- **Konstanten** – z.B. `Integer.MAX_VALUE`
- Darstellung von **Wert des primitiven Typs als Objekt** (später)

Parsen von Zeichenketten: Werte extrahieren

typabhängiges Auslesen von Werten aus String s
mit **Parse**-(=Lese-)Klassenmethode für jeweiligen Typ:

boolean: Boolean.parseBoolean(s) ("true"/"false")

int: Integer.parseInt(s) (opt. VZ, Dezimalziffern)

double: Double.parseDouble(s)
(opt. VZ, Dez.ziffern, opt. Dez.punkt/-komma – u.a.)

s muss gültigen Wert darstellen (**ohne White Space!**)

sc.nextInt() äquivalent formulierbar (und implementiert) als:

```
Integer.parseInt(sc.next()) // nächste Zeichenkette lesen,  
                           // daraus Ganzzahl lesen
```

(entsprechend für nextBoolean(), nextDouble(), ...)

Parzen von Zeichenketten mit Scanner

analog zum Lesen aus `System.in`:

```
12 public static int ersterIntInEingabe() {  
13     // schlechtes Beispiel:  
14     // System.in-Scanner stets in main-Methode anlegen!  
15     Scanner sc = new Scanner(System.in);  
16     int r = ersterInt(sc);  
17     sc.close();  
18     return r;  
19 }
```

kann ein `Scanner` Teile aus einem `String` lesen:

```
20 public static int ersterIntInString(String s) {  
21     // Scanner zu anderen Quellen problemlos lokal anlegbar  
22     Scanner sc = new Scanner(s); // lese aus s statt System.in  
23     int r = ersterInt(sc);  
24     sc.close();  
25     return r;  
26 }
```

Parsen mit Scanner

für Lesevorgang selbst ist Art der Quelle irrelevant:

```
1 | import java.util.Scanner;
   |
   | // ...
3 | public static int ersterInt(Scanner sc) {
4 |     while (sc.hasNext()) {
5 |         if (sc.hasNextInt()) {
6 |             return sc.nextInt();
7 |         }
8 |         sc.next(); // ignorieren, da keine Ganzzahl
9 |     }
10 |    return 0;      // keine Ganzzahl gefunden (besser: Fehler)
11 | }
```


Übung: Eingabe.filter

Aufgabenblatt 04, Aufgabe 08